



UMLNATOR

ENTREGA FINAL

v1.0.0

1. Introducción	2
2. Modelo Computacional	2
2.1. Dominio	2
2.2. Lenguaje	3
3. Implementación	4
3.1. Frontend	4
3.2. Backend	4
3.3. Dificultades encontradas	5
4. Futuras extensiones	5
5. Conclusiones	6
6. Referencias	6

INTEGRANTES

Nombre	Apellido	Legajo	E-mail
Franco	Bonesi	64239	fbonesi@itba.edu.ar
Luca	Rossi	63730	lucarossi@itba.edu.ar
Román	Berruti	63533	rberruti@itba.edu.ar
Matías	Romanato	62072	maromanato@itba.edu.ar

1. Introducción

El conglomerado Elven Door busca optimizar la escalabilidad y consistencia de sus desarrollos. Con la diversidad de equipos de ingeniería y la multiplicidad de tecnologías empleadas, es necesario contar con una solución unificada que permita que todas las aplicaciones se desarrollen sobre una misma base. La actualización centralizada del compilador y del DSL facilitará mejoras en rendimiento, seguridad y prácticas de desarrollo.

En este contexto, "UMLNator" se plantea como un DSL (Domain Specific Language) y su compilador, cuya función principal es recibir código SQL (en particular el referido a la creación de tablas) y, a partir de este, generar el código en PlantUML. Este código podrá ser posteriormente procesado en plataformas como el editor de PlantUML para obtener la representación visual del modelo de datos.

Con esto, se busca facilitar la comprensión de las tablas creadas con código SQL, ya que a partir del código generado en PlantUML y su posterior inserción en un editor, se puede identificar de manera más simple visualmente cómo están conformadas y definidas las tablas, qué atributos hacen referencia a aquellos de otras tablas, qué restricciones fueron planteadas, entre otras características.

2. Modelo Computacional

2.1. Dominio

El dominio que aborda "UMLNator" está diseñado para:

- Entrada: Aceptar especificaciones en SQL, orientadas a la definición de bases de datos, que incluyan la creación de tablas, definición de campos, claves primarias y foráneas y restricciones.
- Salida: Generar código en PlantUML, el cual permite representar:
 - Entidades: Cada tabla del SQL se traduce en una entidad en PlantUML.

- Atributos: Los campos o columnas se convierten en atributos de las entidades correspondientes, cada uno con sus respectivas restricciones si es que fueron definidas.
- Relaciones: Las claves foráneas se mapean a asociaciones entre entidades, donde se pueden expresar las cardinalidades.

Particularmente, el dominio tiene como objetivo:

- Automatización de la Transformación: Facilitar la conversión directa de código SQL a código en PlantUML, eliminando procesos manuales y minimizando la posibilidad de errores, centrándose exclusivamente en la conversión de la creación de tablas, a diferencia de otros DSLs que podrían abordar aspectos más generales del modelado o la simulación.
- Mejora de la Developer Experience (DX): Proveer una herramienta intuitiva que, a partir de un script SQL, genere de forma automática el código que representa la estructura de la base de datos, mejorando y facilitando la comprensión, el diseño y mantenimiento de la misma.

2.2. Lenguaje

El lenguaje ofrece las siguientes funcionalidades:

- Creación y definición de una o más tablas mediante CREATE TABLE.
- Especificación de atributos, tipos de datos y restricciones en la creación de las tablas.
- Definir claves foráneas entre las entidades.
- Indicar que un atributo hace referencia al de otra tabla mediante la palabra clave REFERENCES.
- Ofrecer tipos de datos estándar tales como INT, VARCHAR, BOOLEAN, FLOAT, etc.
- Definir restricciones como PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, etc.
- Definir restricciones como ON DELETE CASCADE, ON DELETE SET NULL, etc.

3. Implementación

3.1. Frontend

En esta sección se empezó a programar la primera parte necesaria para poder llevar a cabo la implementación de UMLNator, teniendo en cuenta el analizador léxico y el sintáctico, ambos necesarios para distintas partes de este proceso.

En primer lugar, para el analizador léxico se utilizó Flex. Así, se tuvieron que definir primero cuáles son los lexemas que deben ser aceptados y procesados teniendo en cuenta la sintaxis propia de creación de tablas en SQL. Para ello, se debieron considerar todos los tipos de datos posibles de definir, constraints, funciones, operadores booleanos y otras palabras reservadas del lenguaje. Luego, se desarrollaron las funciones necesarias para poder convertir el texto ingresado en tokens.

Posteriormente, se hizo uso de Bison para el analizador sintáctico. Se tuvo que definir la gramática del lenguaje con sus respectivos símbolos terminales y no terminales, teniendo en cuenta todas las posibles combinaciones válidas en la creación de tablas. Se desarrollaron los structs necesarios para poder guardar la información de los símbolos no terminales de manera clara y organizada, haciendo uso también de enums para distinguir variaciones de una misma estructura. Finalmente, se desarrollaron las funciones necesarias para poder guardar correctamente toda la información en las estructuras que correspondan reservando memoria si se necesitara, para luego definir las funciones relacionadas a liberar la memoria previamente reservada.

Por último, se desarrollaron los tests necesarios para poder comprobar que funcionaran correctamente casos de aceptación y rechazo del texto de entrada, y así validar que todo lo descrito anteriormente estuviera bien definido para que luego sea procesado por el compilador.

3.2. Backend

La tabla de símbolos se representa como un HashMap anidado, donde la clave es el nombre de la tabla SQL y el valor es otro HashMap que almacena los símbolos de esa tabla. Cada símbolo corresponde a un identificador y contiene información relevante sobre su tipo y atributos.

- El tipo HashMap se define en el archivo HashMap.h y consiste en un arreglo de buckets donde cada uno es una lista enlazada de entradas (Entry).
- Cada entrada (Entry) almacena una clave (key), un valor (value) y un puntero al siguiente elemento de la lista.

```
typedef struct Entry {  
    char * key;  
    void * value;  
    struct Entry * next;  
} Entry;
```

```
typedef struct {  
    Entry * buckets[TABLE_SIZE];  
} HashMap;
```

- Los símbolos individuales se representan mediante la estructura Symbol, definida en SymbolTable.h:

```
typedef struct Symbol {  
    DataType type;  
    DataValue value;  
    boolean isPrimaryKey;  
    boolean isUnique;  
} Symbol;
```

- El valor de un símbolo (DataValue) es una unión que puede almacenar distintos tipos de datos, como enteros, dobles, cadenas o booleanos.

Para manejar distintos ámbitos (scopes) de las tablas, se utiliza una pila (Stack) definida en el archivo Stack.h. Esta pila almacena los nombres de las tablas activas durante el análisis para así generar correctamente la visibilidad de los identificadores.

```
typedef struct {  
    char * items[MAX];  
    int top;  
} Stack;
```

El estado global del compilador se encapsula en la estructura `CompilerState`, definida en `CompilerState.h`. Esta estructura transporta la información relevante a lo largo de las distintas fases del compilador, incluyendo la tabla de símbolos, la pila de scopes, el árbol de sintaxis abstracta (AST) y flags de éxito o error.

```
typedef struct {  
    void * abstractSyntaxTree;  
    boolean succeed;  
    boolean errors;  
    HashMap * symbolTable;  
    Stack * scopeStack;  
} CompilerState;
```

Durante la inicialización del compilador (ver `EntryPoint.c`), se crean e inicializan las estructuras de datos previamente mencionadas. El ciclo de vida del backend incluye la inicialización de módulos, el análisis sintáctico y semántico, y la liberación de recursos al finalizar.

El backend realiza validaciones semánticas sobre el AST generado por el parser. Estas validaciones incluyen:

- Verificación de la existencia y unicidad de atributos.
- Comprobación de compatibilidad de tipos para valores por defecto y referencias.
- Validación de restricciones como claves primarias, claves foráneas, unicidad y checks.
- Manejo de errores semánticos, registrando mensajes claros en caso de inconsistencias.

Las funciones de validación semántica utilizan la tabla de símbolos y la pila de scopes para comprobar la validez de las declaraciones y restricciones en el contexto adecuado.

En cuanto a la generación de código, esta acción comienza con la función `generate`. En primer lugar, se llama a `_generatePrologue()` para generar “@startuml”, que es el comienzo de todo código UML. Luego, se hace uso del `compilerState` para poder iniciar el recorrido del árbol con la función `_generateProgram()`. A partir de este momento, se empieza un llamado recursivo de funciones `generate` para lograr generar el código necesario para cada nodo del árbol, teniendo en cuenta la sintaxis propia de UML.

Dentro de algunas de estas funciones se utilizan otras que se encuentran en el archivo de la tabla de símbolos para poder obtener ciertos datos relevantes directamente de la misma, tales como cuál es el scope actual, el nombre de la tabla de interés (que está relacionada al scope), entre otras.

3.3. Dificultades encontradas

Teniendo en cuenta las tecnologías utilizadas en el proyecto, se presentaron algunas dificultades a lo largo del desarrollo. Respecto a la etapa de frontend, se tuvo que redefinir la gramática en varias ocasiones porque no se estaban teniendo en cuenta algunos casos particulares de la sintaxis. Dada la forma en la que funciona Bison, esto generaba que se tuvieran que modificar reiteradas veces muchas partes distintas del código relacionadas a las estructuras, los nodos y las funciones utilizadas.

En cuanto al backend, se presentaron complicaciones en la generación de código, teniendo en consideración que esto funciona de manera recursiva, llamando a distintas funciones del tipo `generate` según corresponda para poder recorrer el árbol. Hubo ocasiones en las cuales no era simple recuperar algunos datos directamente del árbol porque iban a tener que ser utilizados en otro momento, por lo que debieron programarse funciones específicamente para estos casos y así obtener lo que se necesitaba.

4. Futuras extensiones

Teniendo en cuenta que el objetivo del compilador es convertir las tablas SQL en objetos UML, si se continuara con el trabajo, se podrían validar expresiones aritméticas, “Alter Tables”, y aceptar las funciones más complejas de SQL en DDL.

5. Bibliografía

- Plantuml Language Reference Guide. *PlantUML.com*. (n.d.). Retrieved June 19, 2025, from <https://plantuml.com/es/guide>
- Chapter 5. *data definition*. PostgreSQL Documentation. (2025, May 8). Retrieved June 20, 2025, from <https://www.postgresql.org/docs/17/ddl.html>
- PlantUML editor. (n.d.). Retrieved June 19, 2025, from <https://editor.plantuml.com/uml/>